

Assignment 8 – SQL & PySpark Interview Preparation

Complete interview-preparation notes based on the five questions shared in the images. Each problem is explained conceptually and then solved using SQL and PySpark where applicable.

Question 1 — Find Unique Source–Destination Combinations

Treat $A \rightarrow B$ and $B \rightarrow A$ as the same route.

Input examples:
Delhi → Pune 1400
Pune → Delhi 1400
Bangalore → Chennai 350
Chennai → Bangalore 350

Both directions of the same route must become one canonical pair.

SQL

```
SELECT DISTINCT
    LEAST(Start_Location, End_Location) AS Start_Location,
    GREATEST(Start_Location, End_Location) AS End_Location,
    Distance
FROM routes;
```

LEAST() puts the alphabetically smaller location first and GREATEST() puts the other location second. Therefore Delhi–Pune and Pune–Delhi become the same pair.

PySpark

```
from pyspark.sql.functions import *

result = (
    df
    .withColumn(
        "location_1",
        least(col("Start_Location"), col("End_Location"))
    )
    .withColumn(
        "location_2",
        greatest(col("Start_Location"), col("End_Location"))
    )
    .select(
        col("location_1").alias("Start_Location"),
        col("location_2").alias("End_Location"),
        "Distance"
    )
    .distinct()
)

result.show()
```

Expected:

Delhi	Pune	1400
Bangalore	Chennai	350
Mumbai	Ahmedabad	500
Patna	Ranchi	300

Interview pattern: Undirected pair → canonical representation using LEAST/GREATEST → DISTINCT.

Question 2 — Flag Orders With More Than 2 Failed Payments

Flag an order as Yes when payments for that order have failed more than two times.

Order 101 → 3 Failed → Yes
Order 102 → 0 Failed → No
Order 103 → 2 Failed → No

Important: "more than 2" means > 2 , not ≥ 2 .

SQL

```
WITH failed_payments AS (
    SELECT
        OrderID,
        COUNT(*) AS failed_count
```

```

FROM Payments
WHERE Status = 'Failed'
GROUP BY OrderID
)
SELECT
    o.OrderID,
    o.CustomerID,
    o.OrderDate,
    o.Amount,
    CASE
        WHEN COALESCE(f.failed_count, 0) > 2
        THEN 'Yes'
        ELSE 'No'
    END AS Flag
FROM Orders o
LEFT JOIN failed_payments f
    ON o.OrderID = f.OrderID;

```

PySpark

```

from pyspark.sql.functions import *

failed_payments = (
    payments_df
    .filter(col("Status") == "Failed")
    .groupBy("OrderID")
    .agg(count("*").alias("failed_count"))
)

result = (
    orders_df
    .join(failed_payments, "OrderID", "left")
    .withColumn(
        "failed_count",
        coalesce(col("failed_count"), lit(0))
    )
    .withColumn(
        "Flag",
        when(col("failed_count") > 2, "Yes")
        .otherwise("No")
    )
    .select(
        "OrderID", "CustomerID", "OrderDate", "Amount", "Flag"
    )
)

result.show()

```

Interview pattern: Filter Failed → GROUP BY OrderID → COUNT → join to Orders → apply condition.

Question 3 — Calculate the Percentage of Employees in Each Department

Calculate each department's employee count and its percentage of the total employees.

```

Data Engineering → 4 employees → 40%
Data Analytics   → 3 employees → 30%
Cloud Platform   → 2 employees → 20%
Machine Learning → 1 employee  → 10%

```

Total = 10

SQL

```

SELECT
    department,
    COUNT(*) AS employee_count,
    ROUND(
        COUNT(*) * 100.0 /
        SUM(COUNT(*) OVER ()),
        2
    ) AS percentage_of_total
FROM employees
GROUP BY department;

```

SUM(COUNT(*) OVER ()) gives the grand total of the department counts.

PySpark

```

from pyspark.sql.functions import *

dept_count = (
    df
    .groupBy("department")
    .agg(count("*").alias("employee_count"))
)

total_employees = df.count()

result = (
    dept_count
    .withColumn(
        "percentage_of_total",
        round(
            col("employee_count") * 100 / lit(total_employees),
            2
        )
    )
)

result.show()

```

Interview pattern: GROUP BY → group count → total count → group / total × 100.

Question 4 — Employee and Monthly Sales Analysis

The supplied table contains employee details, departments, managers, salaries, monthly sales and month. The image asks five separate operations.

Q4.1 — Calculate Total Sales and Average Monthly Sales

```

SELECT
    EmployeeID,
    EmployeeName,
    Department,
    SUM(SalesAmount) AS total_sales,
    AVG(SalesAmount) AS average_monthly_sales
FROM employee_sales
GROUP BY
    EmployeeID,
    EmployeeName,
    Department;

```

PySpark

```

employee_sales = (
    df
    .groupBy(
        "EmployeeID",
        "EmployeeName",
        "Department"
    )
    .agg(
        sum("SalesAmount").alias("total_sales"),
        avg("SalesAmount").alias("average_monthly_sales")
    )
)

```

Alice → total 350000, average 175000
 Bob → total 220000, average 110000
 Carol → total 0, average 0
 David → total 0, average 0
 Evan → total 135000, average 67500

Q4.2 — Rank Employees Within Each Department by Total Sales

```

WITH employee_totals AS (
    SELECT
        EmployeeID,
        EmployeeName,
        Department,
        SUM(SalesAmount) AS total_sales
    FROM employee_sales
    GROUP BY EmployeeID, EmployeeName, Department
)
SELECT
    EmployeeID,
    EmployeeName,

```

```

    Department,
    total_sales,
    RANK() OVER (
        PARTITION BY Department
        ORDER BY total_sales DESC
    ) AS department_rank
FROM employee_totals;

```

PySpark

```

from pyspark.sql.window import Window
from pyspark.sql.functions import col, rank

rank_window = (
    Window.partitionBy("Department")
        .orderBy(col("total_sales").desc())
)

ranked_df = employee_sales.withColumn(
    "department_rank",
    rank().over(rank_window)
)

```

Pattern: Rank within a group → PARTITION BY Department + ORDER BY total_sales DESC + RANK().

Q4.3 — Compute a Running Total of Sales Per Department

```

WITH employee_totals AS (
    SELECT
        EmployeeID,
        EmployeeName,
        Department,
        SUM(SalesAmount) AS total_sales
    FROM employee_sales
    GROUP BY EmployeeID, EmployeeName, Department
)
SELECT
    EmployeeID,
    EmployeeName,
    Department,
    total_sales,
    SUM(total_sales) OVER (
        PARTITION BY Department
        ORDER BY total_sales
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM employee_totals;

```

PySpark

```

running_window = (
    Window
        .partitionBy("Department")
        .orderBy("total_sales")
        .rowsBetween(
            Window.unboundedPreceding,
            Window.currentRow
        )
)

result = employee_sales.withColumn(
    "running_total",
    sum("total_sales").over(running_window)
)

Sales department, ordered by total_sales:

Bob    220000 → running total 220000
Alice  350000 → running total 570000

```

Q4.4 — Month-over-Month Comparison

Show the previous month's sales and the difference between the current and previous month.

```

SELECT
    EmployeeID,
    EmployeeName,
    Department,
    Month,
    SalesAmount,
    LAG(SalesAmount) OVER (

```

```

        PARTITION BY EmployeeID
        ORDER BY Month
    ) AS previous_month_sales,
    SalesAmount -
        LAG(SalesAmount) OVER (
            PARTITION BY EmployeeID
            ORDER BY Month
        ) AS difference_from_previous
FROM employee_sales;

```

PySpark

```

month_window = (
    Window
    .partitionBy("EmployeeID")
    .orderBy("Month")
)

result = (
    df
    .withColumn(
        "previous_month_sales",
        lag("SalesAmount").over(month_window)
    )
    .withColumn(
        "difference_from_previous",
        col("SalesAmount") -
        col("previous_month_sales")
    )
)

```

For Alice: January = 150000 and February = 200000, so the February difference is 50000.

Pattern: Previous row/month/value → LAG().

Q4.5 — Filter Departments and Classify Employees

Show departments whose total sales exceed ■100,000. Within those departments, classify an employee as Top Performer if total sales are greater than the department average; otherwise Below Average.

```

WITH employee_totals AS (
    SELECT
        EmployeeID,
        EmployeeName,
        Department,
        SUM(SalesAmount) AS total_sales
    FROM employee_sales
    GROUP BY EmployeeID, EmployeeName, Department
),
department_totals AS (
    SELECT
        Department,
        SUM(total_sales) AS department_total_sales
    FROM employee_totals
    GROUP BY Department
),
filtered_departments AS (
    SELECT *
    FROM department_totals
    WHERE department_total_sales > 100000
),
employee_with_avg AS (
    SELECT
        e.*,
        AVG(e.total_sales) OVER (
            PARTITION BY e.Department
        ) AS department_average
    FROM employee_totals e
    JOIN filtered_departments d
    ON e.Department = d.Department
)
SELECT
    EmployeeID,
    EmployeeName,
    Department,
    total_sales,
    department_average,
    CASE
        WHEN total_sales > department_average
        THEN 'Top Performer'
    END

```

```

        ELSE 'Below Average'
    END AS performance
FROM employee_with_avg;

```

PySpark

```

employee_totals = (
    df
    .groupBy("EmployeeID", "EmployeeName", "Department")
    .agg(sum("SalesAmount").alias("total_sales"))
)

department_totals = (
    employee_totals
    .groupBy("Department")
    .agg(sum("total_sales").alias("department_total_sales"))
    .filter(col("department_total_sales") > 100000)
)

dept_window = Window.partitionBy("Department")

result = (
    employee_totals
    .join(
        department_totals.select("Department"),
        "Department",
        "inner"
    )
    .withColumn(
        "department_average",
        avg("total_sales").over(dept_window)
    )
    .withColumn(
        "performance",
        when(
            col("total_sales") > col("department_average"),
            "Top Performer"
        ).otherwise("Below Average")
    )
)

```

Based on the supplied data, Sales totals 570000 and IT totals 135000, so both exceed 100000. HR totals 0 and is excluded. Evan's IT total equals the IT average, so the strict 'greater than' condition places him in Below Average.

Question 5 — Users With Transactions on at Least 3 Different Dates

Identify users who made transactions on at least three different dates, then calculate their average transaction amount.

```

User 101:
2024-01-01 → 500
2024-01-03 → 300
2024-01-07 → 200

3 distinct dates → qualifies

Average = (500 + 300 + 200) / 3
         = 333.33

User 102 → 2 dates → does not qualify
User 103 → 2 dates → does not qualify

```

SQL

```

SELECT
    user_id,
    AVG(transaction_amount) AS average_transaction_amount
FROM transactions
GROUP BY user_id
HAVING COUNT(DISTINCT transaction_date) >= 3;

```

PySpark

```

from pyspark.sql.functions import *

result = (
    df
    .groupBy("user_id")
    .agg(
        countDistinct("transaction_date").alias("distinct_dates"),

```

```

        avg("transaction_amount").alias(
            "average_transaction_amount"
        )
    )
    .filter(
        col("distinct_dates") >= 3
    )
    .select(
        "user_id",
        "average_transaction_amount"
    )
)

result.show()

Expected:
user_id  average_transaction_amount
101      333.33

```

Why HAVING?

WHERE filters individual rows before grouping. HAVING filters groups after aggregation. Since COUNT(DISTINCT transaction_date) is an aggregate, the condition belongs in HAVING.

Assignment 8 — Interview Pattern Summary

```

Q1 → LEAST() + GREATEST() + DISTINCT
Q2 → Conditional aggregation + CASE WHEN
Q3 → GROUP BY + total + percentage
Q4.1 → GROUP BY + SUM / AVG
Q4.2 → PARTITION BY + RANK
Q4.3 → Window SUM
Q4.4 → LAG
Q4.5 → Window AVG + CASE WHEN
Q5 → COUNT(DISTINCT) + HAVING

```

Interview lesson: Before writing code, identify the data pattern. Then choose the SQL/PySpark operation that matches that pattern.